

学习笔记0625

SIPU硬件总体架构

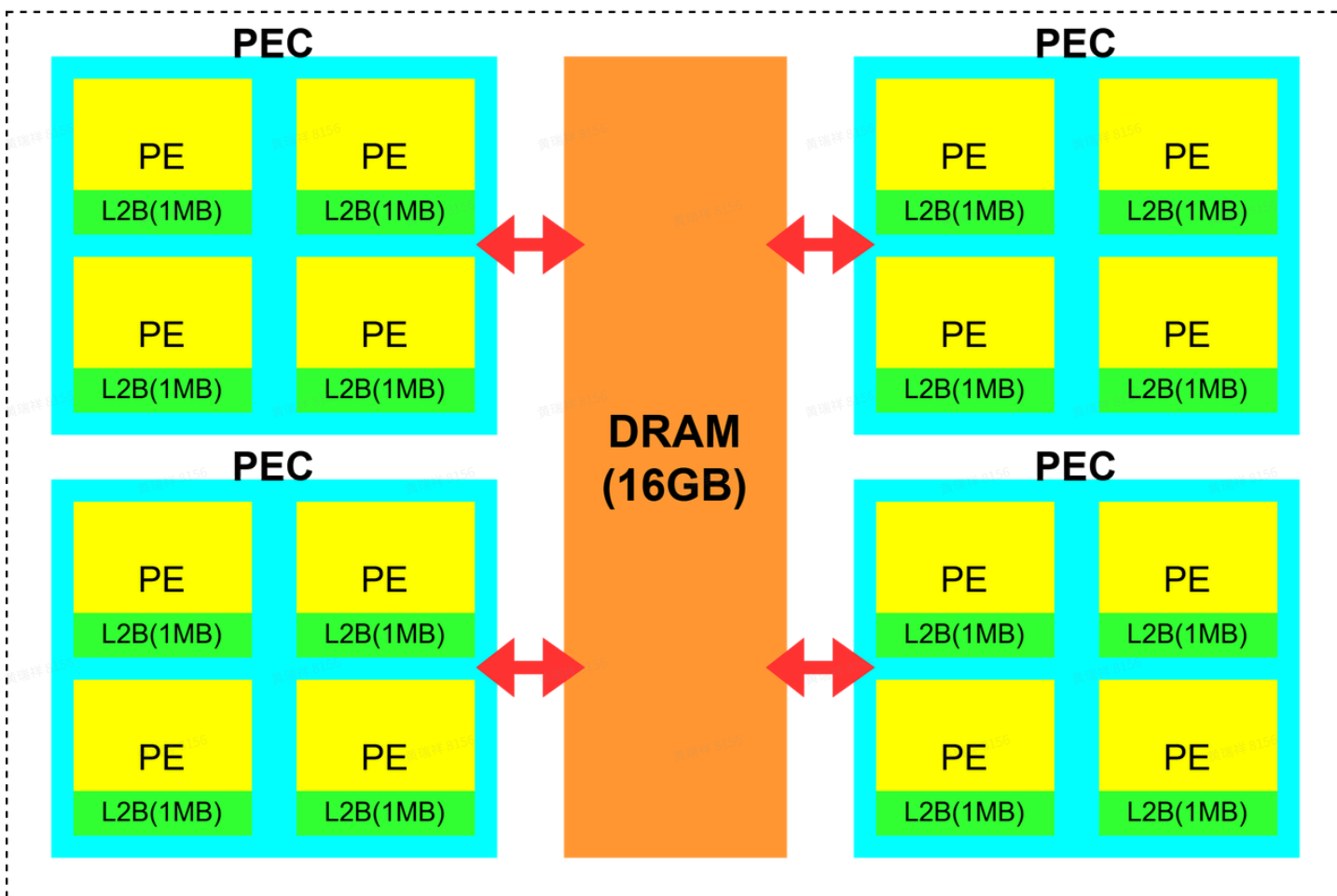
SIPU硬件总体架构如图：总体分为两个PEG（Group），每个PEG包含四个PEC（Cluster），每个PEC包含四个PE，总共32个PE。每个PE配备1MB的L2B，作为shared memory，每个PEC配备4GB的DRAM，四个PEC的DRAM interleave为一个16GB的整体，作为global memory。

- L2B即L2 Buffer，由SRAM做成，挂在PE旁边的本地存储区，降低全局带宽需求，也降低延迟，也节省功耗，SRAM比DRAM省电。L2B是一种显式管理的数据缓冲区，可能由DMA、编译器或者调度器控制，而不是像CPU cache那样完全自动缓存。Level-2 Buffer是一种“显式缓冲存储”，是片上的SRAM暂存区，需要程序员手动管理。加速器的计算模式比较规则，数据访问模式往往可以提前知道，可以做得比自动cache更可控，不用像CPU那样考虑通用性和自动化。
- L2C即L2 Cache，是一种“缓存机制”，由硬件自动管理，程序员无需显示控制和Cache的交互，只需要按编址访问即可。cache line是cache读写、命中、替换的基本单位。cache line冲突指的就是不同硬件单元同时访问同一个cache line，此时需要进行额外的同步操作（invalid-flush-reload），性能就会下降。
- L2B和L2C都位于存储层级的第二级，所以叫L2。硬件系统会把存储分为很多级，按照离计算单元从近到远分为：寄存器、L1、L2、L3（Global Buffer）、DRAM/HBM。
- interleave即交错编址，把多个DRAM组合成一个更大的逻辑内存空间。interleave的颗粒度为4KB，意思是每4KB轮流放到不同DRAM里。interleave的目的，是提高带宽，避免单个DRAM成为瓶颈，解决带宽利用不均匀的问题，同时一个访问能同时利用多个DRAM的带宽。

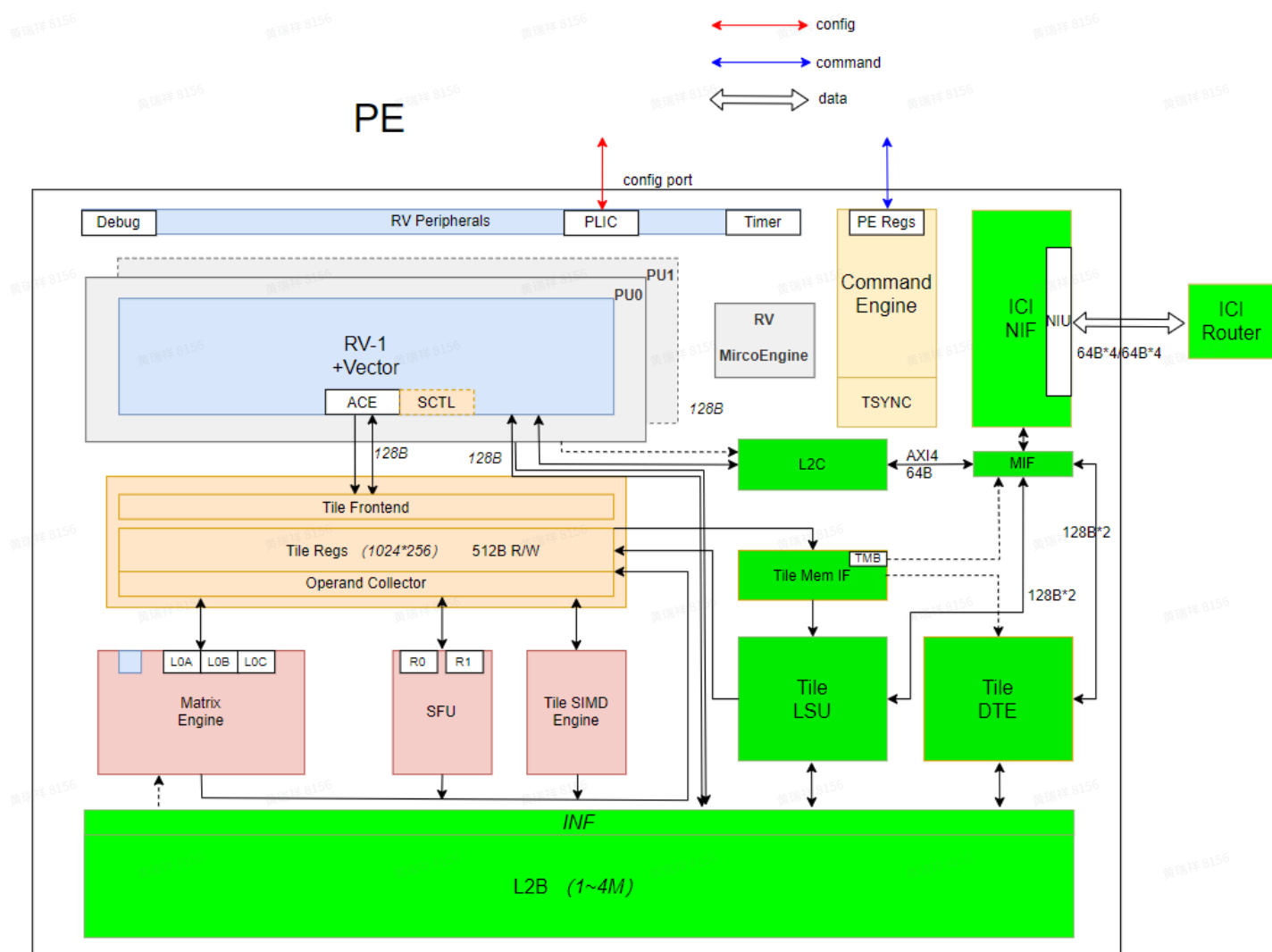
单个算子可调度的范围为一个PEG，SIPU kernel各个维度可配置的情况为：grid（1-4096）、cluster（1-4）、block（1-2）



PEG



每个PE包含两个RV Core和一个Tile Core，共享L2B；L2C的cache line为64Byte。RV Core scalar整型寄存器数量：32个reg（64 bit）、RV Core scalar浮点寄存器数量：32个reg（64 bit）、RV Core Vector寄存器数量：32个vec（1024 bit）、Tile Core Vector寄存器数量：256个TReg（1024 Byte）。此外，每个PE还包含TLSU，TALU，DTE等若干硬件单元。程序以线程为单位执行，多个线程共享一个PE时，需要配置这些寄存器怎么分配到这些线程上。



grid/cluster/block和PEG/PEC/PE之间的对应关系是：grid为逻辑概念，不对应硬件，griddim在这里的含义为总共有多少block要执行，相当于任务总数，不是实际硬件数量，最多可设置4096；cluster对应PEC，clusterdim为cluster内启用的block数量，可设置1-4；block对应PE，blockdim为block内thread数量，由于只有两个RV Core，可设置1-2；

程序开发工具siorigin_tile

算子开发依赖runtime提供的基础功能API和compiler提供的Tile Core和RV Core API。目前kernel开发主要使用的指令为Tile Core builtin指令集合，与RV Core builtin指令集合。

runtime封装提供的编译工具为scc，用法和gcc类似；除了直接使用编译工具进行编译，更推荐使用CMake构建工具集成管理

```

1  cmake_minimum_required(VERSION 3.27)
2  project(MySipuApp)
3
4  find_package(scc REQUIRED)
5
6  scc_add_executable(v vectMul.su v1.su)
7  scc_target_include_directories(v ./include1 ./include2)
8  scc_target_compile_definitions(v ENABLE_DEVICE_KERNEL_LAUNCH=1)
9  scc_target_compile_options(v "-O0" "-Wall")
10 scc_install(v DESTINATION bin)
11
12 scc_add_library(v2 SHARED v2.cpp)
13 scc_target_compile_options(v2 "-O2")
14 scc_install(v2 DESTINATION lib)

```

编译器标识符和CUDA类似，也有__global__、__device__、__shared__。

编译器会把源代码翻译为目标平台的代码，disable_tile_scheduling可以作用在global或device函数上，编译器对这些函数会尽量使汇编中的Tile指令采用源代码的顺序，这称为Tile指令保序。

为了提高指令执行效率，也会通过手写汇编代码（这里就是Tile Core和RV Core的指令）来对指令流水进行编排（手动调整指令顺序让各个功能单元一直有活儿干），对寄存器的使用进行展开（多用一些寄存器保存中间值，减少数据依赖），来做底层性能优化。

builtin指令指编译器内建指令，是源代码层面给编译器认识的一类特殊接口；而汇编指令是ISA（指令集架构）层面的具体指令。builtin指令的作用就是能在源代码层面调用一些源代码常规语法表达不了但是硬件支持的能力。

SIPU编程模型是典型的 host-device 异构计算模型，包含主机（host）端代码，和设备（device）端代码；算子的核函数（kernel）中可通过配置grid/cluster/block来指派多个线程。一个数组相加的例子如下

代码块

```

1  #include <sipu_runtime.h>
2
3  __global__ void add(const int* A, const int* B, int* C, int n) {
4      uint32_t id = blockDim.x * blockIdx.x + threadIdx.x;
5      uint32_t step = blockDim.x * gridDim.x;
6
7      for(uint32_t i = id; i < n; i+=step) {
8          C[i] = A[i] + B[i];
9      }
10 };
11
12 #define N 1024
13 int main(int argc, char* argv[]) {

```

```

14     size_t dsize = N * sizeof(int);
15     int host_A[N];
16     int host_B[N];
17     int host_C[N];
18
19     int* data_a = nullptr;
20     int* data_b = nullptr;
21     int* data_c = nullptr;
22     sipuMalloc(&data_a, dsize);
23     sipuMalloc(&data_b, dsize);
24     sipuMalloc(&data_c, dsize);
25
26     sipuMemcpy(data_a, host_A, dsize, sipuMemcpyHostToDevice);
27     sipuMemcpy(data_b, host_B, dsize, sipuMemcpyHostToDevice);
28
29     dim3 grid{16};
30     dim3 block{2};
31     add<<<grid, block>>>(data_a, data_b, data_c, N);
32
33     sipuMemcpy(host_C, data_c, dsize, sipuMemcpyDeviceToHost);
34
35     sipuFree(data_a);
36     sipuFree(data_b);
37     sipuFree(data_c);
38
39     return 0;
40 }

```

开发特性

如果循环中同时包含访存与计算，那么可以通过减少访存与计算的相关依赖。相关依赖分真依赖和假依赖。真依赖是数据上的实际依赖，例如 `reduce(v1, v2)` 必须等待 `v1 = tmv(...)` 完成，因为计算确实需要 `v1` 的结果，这类依赖无法消除；而假依赖则是由于多次迭代复用了同一个寄存器（如 `v1`）导致的寄存器名冲突。通过寄存器展开（例如将 `v1` 扩展为 `vvec[0]~vvec[7]`），可以消除这种假依赖，使访存单元在当前数据计算时提前加载下一批数据，从而提高访存与计算的流水并发能力。

为了避免cache line冲突，Tile Core不要主动访问kernel代码内的栈空间，翻译过来就是，不要把局部变量地址直接传给Tile Load/Store。

RAS（Return Address Stack）是RV Core的函数调用预测硬件，只有4级深度，函数调用层次太深时，返回地址预测会失效，导致流水线被打断，性能下降，因此编程时函数调用不要超过这个深度，只调用少数几次或较短的函数都建议用inline标记。

RV Core的分支预测器能力有限，没有间接跳转预测器，只有直接跳转预测器（JAL, jump and link）。switch-case编译出来的间接跳转预测很差，少用。if else语句也尽量用于判断thread id等，使得每次预测结果都是同一个结果，分支预测期很容易学。

在AI加速器的类型系统里，m1/m2/m4/m8表示一个对象占用了多少倍的基础向量/tile资源，也即寄存器宽度倍数。union是C/C++里的联合体，所有成员共用同一块内存，同一时刻本质上只有一种解释方式；这里可以用来应对数据访存支持m2级别操作但是运算操作不支持的尴尬情况

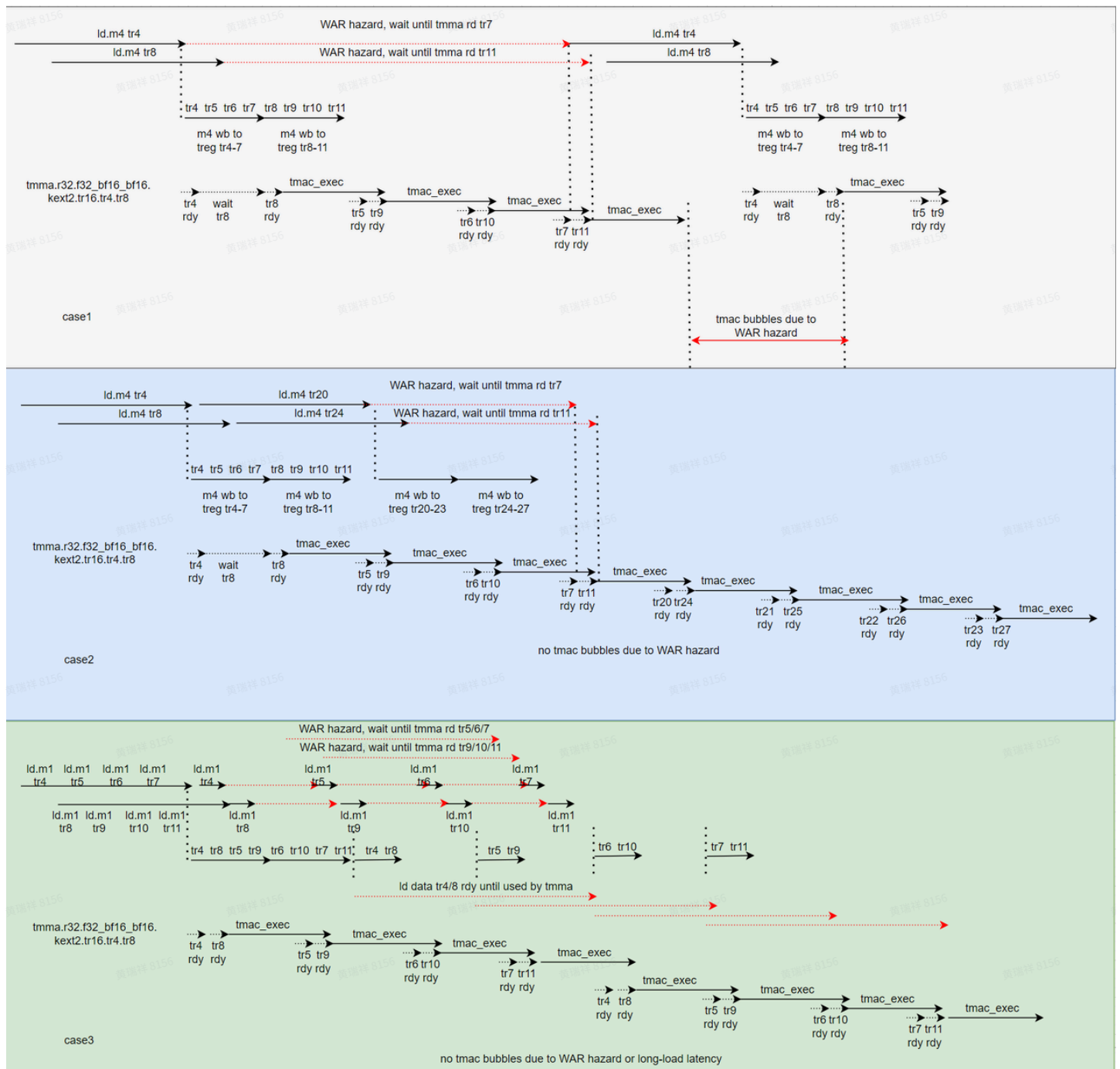
推测执行本质上就是：在控制流（走哪条分支）还没最终确定之前，先把大概率会执行的指令拿进流水线执行（预取-译码-发射-执行，只是没提交执行结果）。能推测执行的通常是普通计算指令（加减乘除、地址计算等）；不能推测执行的通常是会访问特殊硬件状态、发起复杂访存、或者产生不可轻易撤销副作用的指令。普通计算指令能推测执行的本质，就是这种指令预取到流水线里，即使猜错了副作用也很小。如果一条Tile Core指令需要15 cycles，执行到中间发现分支猜错了，Tile已经开始算了，这时到底是中断/回滚/清空，情况就变得复杂了。类似这种涉及硬件状态、资源占用或不可轻易撤销的副作用的指令，通常不会被推测执行。比如涉及CSR（Control and Status Register，控制状态寄存器）操作的指令都不适合推测执行。

循环体和函数内，标量指令尽量密集放下靠前的位置，RVV/Tile Core和CSR操作是不能推测执行的。位置无关的指令建议多放在代码前面的位置。另外每个循环尽量大些来平摊掉分支预测产生的bubble，分支预测器本身需要几个cycle来产生预测结果，并且循环内尽量不要用到跳转指令。换个说法，不要把Tile指令写在小循环里，应该尽量把循环展开，让一次跳转对应更多运算，填补Tile指令的长延迟。

tadd产生的寄存器依赖需要14个cycle，意思是tadd的结果写回Tile寄存器后，要过14cycle，后续依赖这个结果的指令才能安全读取，这叫latency（结果可用延迟）；所以两者至少要隔开14cycle，或者插入足够多的无关指令来填充这段延迟。

如果只是把一个Tile寄存器的数据拷贝到另一个Tile寄存器，优先用tadd加零，而不是tmv。tmv走的是ACE流水线，发射效率只能达到1 inst/cycle（每个cycle只能发射一条流水线）；而tadd走的是TALU流水线，TALU的吞吐率更高，相较于ACE没有那么容易成为瓶颈。

算子kernel代码需要注意load/store/mma指令的配比，某一类指令不能太多，一类指令太多可能导致指令发不下去。load/store建议使用M1，硬件解依赖M1最快（硬件对存在依赖关系的数据是否准备好的判断，称为解依赖，硬件内部会有scoreboard或hazard detector之类的电路来自动判断，准备好则发射下一条指令，否则等待），如下图三个case流程图分析：



地址对齐简单来说就是，硬件希望数据从某个固定大小额边界开始存放而不是从任意地址开始。首地址对齐指的就是数据块第一个字节的地址。例如要求32B对齐，意思是首地址必须是32的整数倍。因为硬件访问数据是一次读一整块。不对齐的情况下，一次访问会由于跨块而变成两次。Linear一次搬一个RV Vector 32B对齐，Tiled一次搬整个Tile的一部分 256B对齐

BF16和FP16都是16位浮点数，位分配不同；float是1-5-10的分配（符号-指数-尾数），精度高范围小；bfloat16是谷歌在做TPU时发现神经网络不需要那么多尾数但需要很大的指数范围，于是改成了1-8-7，指数范围和FP32一样。后来Intel/AMD/NVIDIA等各大厂商的AI芯片都大量支持BF16。

TF32是NVIDIA在Ampere架构中提出的专用于AI计算的浮点格式，在FP32的基础上把尾数减少到10bit，剩的都是无效bit。NVIDIA发现神经网络其实不需要那么多位尾数。计算时只需要有效bit参与运算，因此吞吐率得以提高。

IDE的自动补全是怎么工作的，Tile Core指令为什么看起来像编译器认识的关键字。Clangd本身不懂Tile Core指令，它只调用Clangd编译器前端来分析代码，之所以能自动提示Tile Core指令，是因为

这些指令在 SIPU SDK 的头文件里被声明成了 builtin/intrinsic 函数，而 clangd 能读取这些声明。Clangd 可以理解为 Clang Compiler 的代码分析服务部分，而编译器前端可以识别这些关键字。Clang 本质上是一个 C/C++ 编译器前端，属于 LLVM 项目的一部分。LLVM 项目……，包含 Clang（C/C++ 编译器前端）、LLVM IR（中间表示）、优化器、各种后端（x86/ARM/RISC-V）。gcc/g++ 和 clang/clang++ 都是编译器，都能编译 C++，前者是 GNU 项目、后者是 LLVM 项目。LLVM 的错误提示更友好、IDE 支持非常强，也比较容易定制，很多 AI 芯片厂商会选择 LLVM/Clang 做二次开发。

栈（stack）、堆（heap）、全局区（data/bss）本质上都是地址空间中的一段区域，它们最终都映射到真实内存（DRAM）中，区别主要是管理方式不同。局部变量运行时通常放在栈空间，运行结束后栈自动回收；与之对应的是全局变量存放的堆空间，是手动申请的内存。之所以会产生栈访问，是因为局部变量放进寄存器会发生溢出，所以会让编译器生成先保存到栈空间之后再取回来的指令。

sipu 支持 Cooperative Launch，是类似 CUDA Distributed Shared Memory 的特性，实现了同一个 cluster 内 PE 间的 share mem 互访。对于本 PE 内的 share mem 可以直接访问，其他 PE 的则需要用地址 map API 转换后访问。

DTE 可应用于多种场景，以提高数据搬运与运算的并行效率。Linear 数据搬运，只需要指定源数据位置（如 global memory），目标数据位置（如 shared memory），以及数据大小（字节数）即可。Tile 数据搬运，搬运的内容为 Tile Format 数据，需要结合 tensor map 结构与坐标信息进行操作。DTE 还实现了 Linear 和 Tile 两种排布方式的转换功能。

SIPU 内，存在多种类型的指令流水，且相互独立，因此在单个线程内部，也需要做好多个指令流水的同步操作。不同硬件执行单元（RV Core/TLSU，load and store/TALU/TMAC/TSFU, special func/TDTE, data transfer），各自都有自己独立的流水线（pipeline）。不同类型的指令会进入不同流水线，并且不同流水线之间是相互独立的。然而数据不是独立的，因此需要对不同流水线做同步，保证一个流水线使用的数据已经被另一个流水线准备好了。（线程内同步/单线程同步）；相应的，也需要多线程同步/指令同步

对于 SIPU 内多个 pipeline 的访存控制，也需要进行同步，有一些对应的指令进行访存同步。